

CS 302.1 - Automata Theory

Lecture 12

Shantanav Chakraborty

Center for Quantum Science and Technology (CQST)

Center for Security, Theory and Algorithms (CSTAR)

IIIT Hyderabad



Quick Recap

Recursive Language/Turing Decidable/Decidable: A language L is called Recursive or Turing decidable or Decidable if there exists a Total Turing Machine M and

$$\left. \begin{array}{l} \forall \omega \in L, M(\omega) \text{ accepts} \\ \forall \omega \notin L, M(\omega) \text{ rejects} \end{array} \right\} \text{Halts on all inputs}$$

The Church Turing thesis: An algorithm can be written for a problem if and only if it is decidable, i.e. there exists a Total Turing machine that solves the problem. **Total TM $\Leftrightarrow$ Algorithms!**

Recursively Enumerable Language/Turing Recognizable (RE): A language L is called Recursively Enumerable (RE) or Turing Recognizable if

$$\begin{array}{ll} \forall \omega \in L, M(\omega) \text{ accepts} & \\ \forall \omega \notin L, M(\omega) \text{ doesn't accept} & \text{(rejects or runs infinitely)} \end{array}$$

Co-Recursively Enumerable Language/co-Turing Recognizable (Co-RE/ $\overline{RE}$ /nRE): A language L is **Co-Recursively Enumerable (co-RE/ $\overline{RE}$)** or **Co-Turing Recognizable** if

$$\begin{array}{ll} \forall \omega \in L, M(\omega) \text{ doesn't reject} & \text{(accepts or loops)} \\ \forall \omega \notin L, M(\omega) \text{ rejects} & \end{array}$$

Quick Recap

There exists a one-one mapping (bijective relationship) between the set of finite length binary strings and TMs.

Universal Turing Machine: A Universal Turing Machine, denoted as U_{TM} accepts as input (i) the encoding of a Turing Machine M and (ii) an input string w and **simulates M running on w** , i.e.

$$U_{TM}(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ accepts} \\ \text{REJECTS, if } M(w) \text{ rejects} \\ \text{LOOPS INFINITELY, if } M(w) \text{ loops infinitely} \end{cases}$$

Quick Recap

There exists a one-one mapping (bijective relationship) between the set of finite length binary strings and TMs.

Universal Turing Machine: A Universal Turing Machine, denoted as U_{TM} accepts as input (i) the encoding of a Turing Machine M and (ii) an input string w and **simulates M running on w** , i.e.

$$U_{TM}(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ accepts} \\ \text{REJECTS, if } M(w) \text{ rejects} \\ \text{LOOPS INFINITELY, if } M(w) \text{ loops infinitely} \end{cases}$$

Some examples of languages that are recursive/decidable:

- $A_{DFA} = \{\langle DFA \rangle, w \mid w \in L(DFA)\}$
- $E_{DFA} = \{\langle DFA \rangle \mid L(DFA) = \Phi\}$
- $A_{CFG} = \{\langle CFG, w \rangle \mid w \in L(CFG)\}$
- $E_{CFG} = \{\langle CFG \rangle \mid L(CFG) = \Phi\}$

An undecidable language:

- $A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$

Quick Recap

There exists a one-one mapping (bijective relationship) between the set of finite length binary strings and TMs.

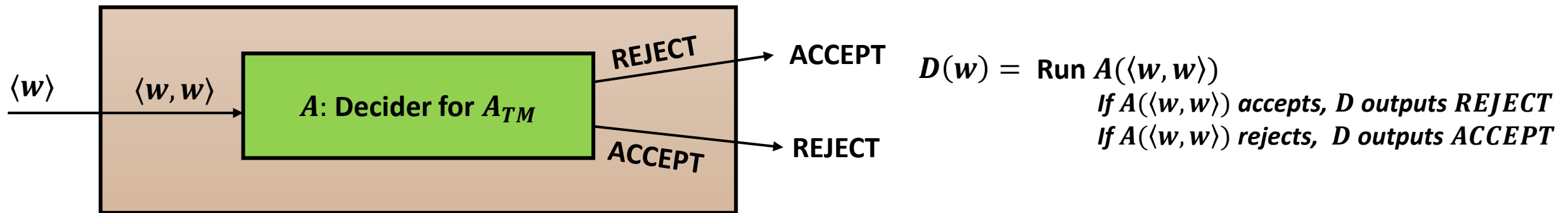
An undecidable language:

- $A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

Proof strategy: By contradiction. We assume that a Total TM A exists that decides A_{TM} .

We build a Total TM D that accepts w as input and calls $A(\langle w, w \rangle)$ as a subroutine and outputs the opposite of A .



Quick Recap

There exists a one-one mapping (bijective relationship) between the set of finite length binary strings and TMs.

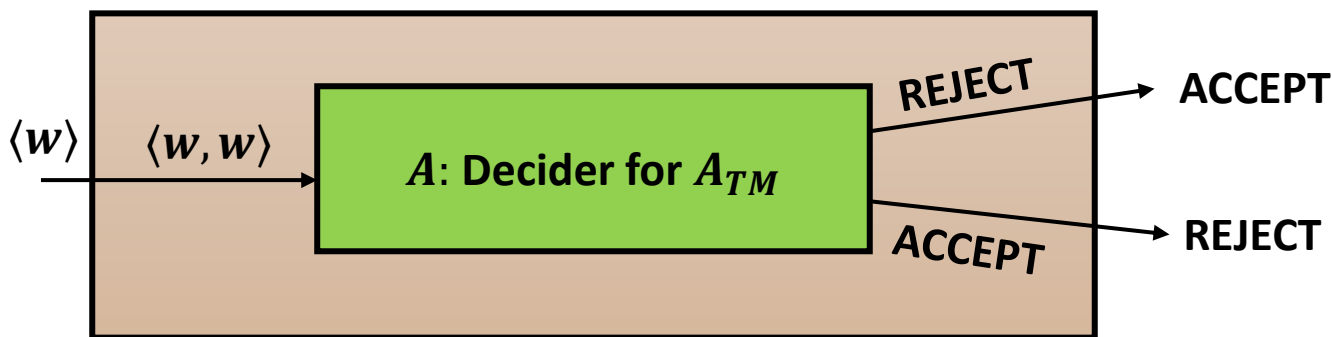
An undecidable language:

- $A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

Proof strategy: By contradiction. We assume that a Total TM A exists that decides A_{TM} .

We build a Total TM D that accepts w as input and calls $A(\langle w, w \rangle)$ as a subroutine and outputs the opposite of A .



For $w = \langle M_w \rangle$

$D(\langle M_w \rangle) =$

Run $A(M_w, \langle M_w \rangle)$

$A(M_w, \langle M_w \rangle)$ accepts, if $M_w(\langle M_w \rangle)$ **accepts**
(D outputs **REJECT**)

$A(M_w, \langle M_w \rangle)$ rejects, if $M_w(\langle M_w \rangle)$ **doesn't accept**
(D outputs **ACCEPT**)

Quick Recap

There exists a one-one mapping (bijective relationship) between the set of finite length binary strings and TMs.

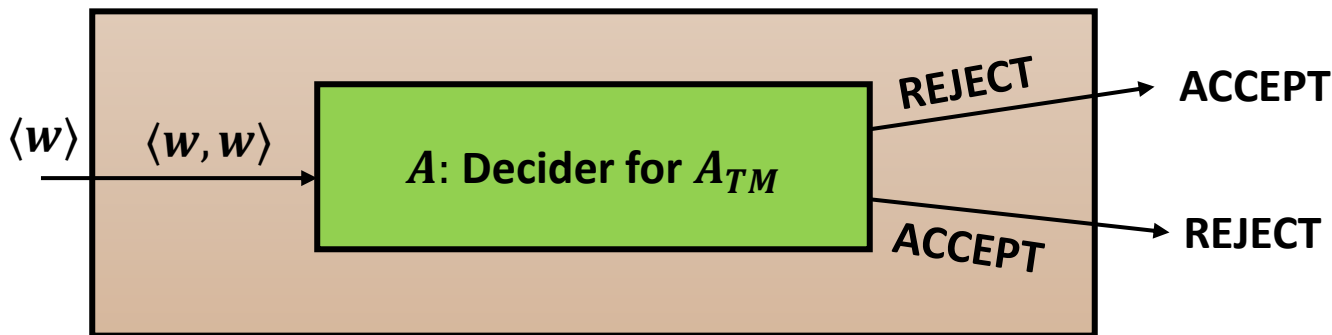
An undecidable language:

- $A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

Proof strategy: By contradiction. We assume that a Total TM A exists that decides A_{TM} .

We build a Total TM D that accepts w as input and calls $A(\langle w, w \rangle)$ as a subroutine and outputs the opposite of A .



For $w = \langle D \rangle$, there is a contradiction!

$D(\langle M_w \rangle) =$

Run $A(M_w, \langle M_w \rangle)$

$A(M_w, \langle M_w \rangle)$ accepts, if $M_w(\langle M_w \rangle)$ **accepts**
(D outputs **REJECT**)

$A(M_w, \langle M_w \rangle)$ rejects, if $M_w(\langle M_w \rangle)$ **doesn't accept**
(D outputs **ACCEPT**)

Quick Recap

There exists a one-one mapping (bijective relationship) between the set of finite length binary strings and TMs.

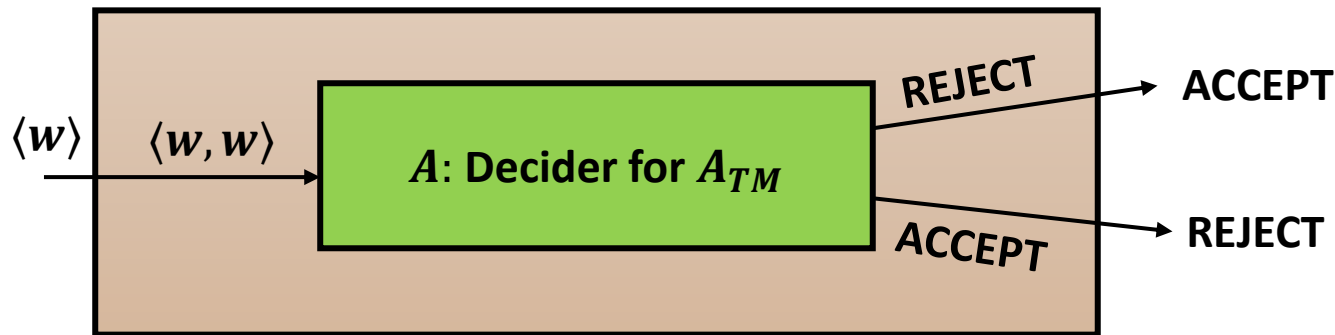
An undecidable language:

- $A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

Proof strategy: By contradiction. We assume that a Total TM A exists that decides A_{TM} .

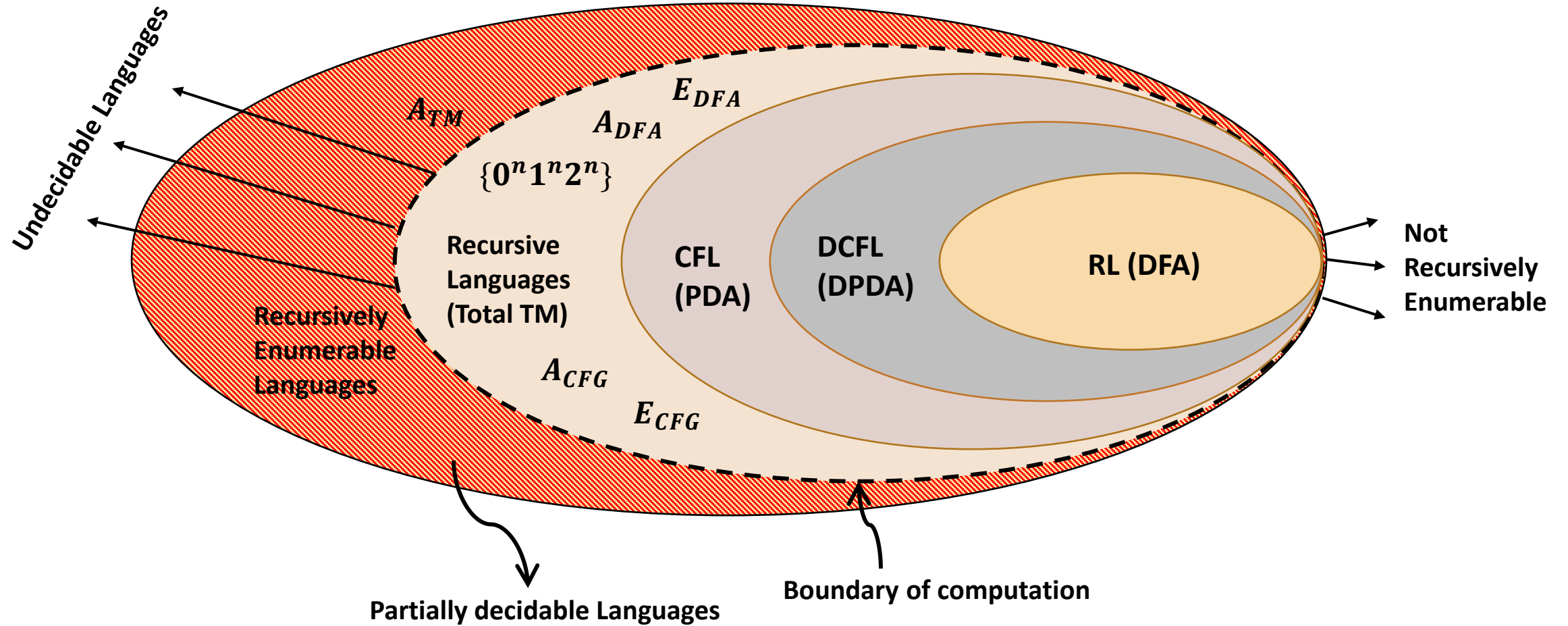
We build a Total TM D that accepts w as input and calls $A(\langle w, w \rangle)$ as a subroutine and outputs the opposite of A .



$D(\langle D \rangle)$ accepts $\leftrightarrow D(\langle D \rangle)$ **doesn't accept**
 $D(\langle D \rangle)$ rejects $\leftrightarrow D(\langle D \rangle)$ **accepts**

For $w = \langle D \rangle$, there is a contradiction!

Quick Recap



An undecidable problem

$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$. A_{TM} is undecidable

$$A(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ accepts} \\ \text{REJECTS, if } M(w) \text{ rejects or loops infinitely} \end{cases}$$

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

The proof uses a technique called **Diagonalization**.

First, recall that there exists a bijective map (one-one correspondence) between the set of all finite-length binary strings and Turing Machines.

We can list all the Turing Machines and write down the result of running any M_i on input $\langle M_j \rangle$.

An undecidable problem

$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$. A_{TM} is undecidable

$$A(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ accepts} \\ \text{REJECTS, if } M(w) \text{ rejects or loops infinitely} \end{cases}$$

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

The proof uses a technique called **Diagonalization**.

	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_0	Accept	Accept	Loops	Reject	Accept	...
M_1	Accept	Reject	Reject	Accept	Reject	...
M_2	Reject	Loops	Accept	Loops	Accept	...
M_3	Accept	Reject	Reject	Accept	Reject	...
M_4	Accept	Accept	Accept	Accept	Reject	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...

First, recall that there exists a bijective map (one-one correspondence) between the set of all finite-length binary strings and Turing Machines.

We can list all the Turing Machines and write down the result of running any M_i on input $\langle M_j \rangle$.

An undecidable problem

$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$. A_{TM} is undecidable

$$A(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ accepts} \\ \text{REJECTS, if } M(w) \text{ rejects or loops infinitely} \end{cases}$$

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

The proof uses a technique called **Diagonalization**.

	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_0	Accept	Accept	Loops	Reject	Accept	...
M_1	Accept	Reject	Reject	Accept	Reject	...
M_2	Reject	Loops	Accept	Loops	Accept	...
M_3	Accept	Reject	Reject	Accept	Reject	...
M_4	Accept	Accept	Accept	Accept	Reject	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...

How would this Table look for A ?

An undecidable problem

$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$. A_{TM} is undecidable

$$A(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ accepts} \\ \text{REJECTS, if } M(w) \text{ rejects or loops infinitely} \end{cases}$$

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

The proof uses a technique called **Diagonalization**.

How would this Table look for A ?

	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_0	Accept	Accept	Loops	Reject	Accept	...
M_1	Accept	Reject	Reject	Accept	Reject	...
M_2	Reject	Loops	Accept	Loops	Accept	...
M_3	Accept	Reject	Reject	Accept	Reject	...
M_4	Accept	Accept	Accept	Accept	Reject	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...

A	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_0	Accept	Accept	Reject	Reject	Accept	...
M_1	Accept	Reject	Reject	Accept	Reject	...
M_2	Reject	Reject	Accept	Reject	Accept	...
M_3	Accept	Reject	Reject	Accept	Reject	...
M_4	Accept	Accept	Accept	Accept	Reject	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...

An undecidable problem

$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$. A_{TM} is undecidable

The proof uses a technique called **Diagonalization**.

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

A	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_0	Accept	Accept	Reject	Reject	Accept	...	Accept	...
M_1	Accept	Reject	Reject	Accept	Reject	...	Accept	...
M_2	Reject	Reject	Accept	Reject	Accept	...	Accept	...
M_3	Accept	Reject	Reject	Accept	Reject	...	Reject	...
M_4	Accept	Accept	Accept	Accept	Reject	...	Reject	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	...
D						...		...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	...

$$D(w) = \{ \text{Run } A(\langle w, w \rangle) \}$$

If $A(\langle w, w \rangle)$ accepts, then **REJECT**

If $A(\langle w, w \rangle)$ rejects, then **ACCEPT**

}

$$D(\langle M_w \rangle) = \begin{cases} \text{ACCEPTS, if } M_w(\langle M_w \rangle) \text{ doesn't accept} \\ \text{REJECTS, if } M_w(\langle M_w \rangle) \text{ accepts} \end{cases}$$

- Somewhere we will also have the TM D .
- Note that D by definition **computes the opposite of the diagonal entries** of the table.

An undecidable problem

$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$. A_{TM} is undecidable

The proof uses a technique called **Diagonalization**.

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

Note that D by definition **computes the opposite of the diagonal entries** of the table.

A	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_0	Accept	Accept	Reject	Reject	Accept	...	Accept	...
M_1	Accept	Reject	Reject	Accept	Reject	...	Accept	...
M_2	Reject	Reject	Accept	Reject	Accept	...	Accept	...
M_3	Accept	Reject	Reject	Accept	Reject	...	Reject	...
M_4	Accept	Accept	Accept	Accept	Reject	...	Reject	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	...
D	Reject					...		...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	...

An undecidable problem

$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$. A_{TM} is undecidable

The proof uses a technique called **Diagonalization**.

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

Note that D by definition **computes the opposite of the diagonal entries** of the table.

A	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_0	Accept	Accept	Reject	Reject	Accept	...	Accept	...
M_1	Accept	Reject	Reject	Accept	Reject	...	Accept	...
M_2	Reject	Reject	Accept	Reject	Accept	...	Accept	...
M_3	Accept	Reject	Reject	Accept	Reject	...	Reject	...
M_4	Accept	Accept	Accept	Accept	Reject	...	Reject	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	...
D	Reject	Accept				...		...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	...

An undecidable problem

$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$. A_{TM} is undecidable

The proof uses a technique called **Diagonalization**.

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

Note that D by definition **computes the opposite of the diagonal entries** of the table.

What will be the D^{th} diagonal entry??

A	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_0	Accept	Accept	Reject	Reject	Accept	...	Accept	...
M_1	Accept	Reject	Reject	Accept	Reject	...	Accept	...
M_2	Reject	Reject	Accept	Reject	Accept	...	Accept	...
M_3	Accept	Reject	Reject	Accept	Reject	...	Reject	...
M_4	Accept	Accept	Accept	Accept	Reject	...	Reject	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	...
D	Reject	Accept	Reject	Reject	Accept	...		...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	...

An undecidable problem

$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$. A_{TM} is undecidable

The proof uses a technique called **Diagonalization**.

- A_{TM} is undecidable
- $A_{TM} \in RE$ but not recursive
- A_{TM} is partially decidable

Note that D by definition **computes the opposite of the diagonal entries** of the table.

What will be the D^{th} diagonal entry??

A	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_0	Accept	Accept	Reject	Reject	Accept	...	Accept	...
M_1	Accept	Reject	Reject	Accept	Reject	...	Accept	...
M_2	Reject	Reject	Accept	Reject	Accept	...	Accept	...
M_3	Accept	Reject	Reject	Accept	Reject	...	Reject	...
M_4	Accept	Accept	Accept	Accept	Reject	...	Reject	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	...
D	Reject	Accept	Reject	Reject	Accept	...	??	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	...

Contradiction!

The Halting problem

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$. Is $HALT_{TM}$ decidable?

The Halting Problem: Does there exist a Total Turing Machine H that accepts as input a Turing Machine M and an input string w and outputs YES, if $M(w)$ halts (accepts or rejects) and NO, if $M(w)$ does not halt (loops forever), i.e.

$$H(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ HALTS, i.e. accepts or rejects} \\ \text{REJECTS, if } M(w) \text{ does not HALT, i.e. loops infinitely} \end{cases}$$

The Halting problem

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$. Is $HALT_{TM}$ decidable?

The Halting Problem: Does there exist a Total Turing Machine H that accepts as input a Turing Machine M and an input string w and outputs YES, if $M(w)$ halts (accepts or rejects) and NO, if $M(w)$ does not halt (loops forever), i.e.

$$H(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ HALTS, i.e. accepts or rejects} \\ \text{REJECTS, if } M(w) \text{ does not HALT, i.e. loops infinitely} \end{cases}$$

- Turing stated the Halting problem and demonstrated its undecidability in his famous 1936 paper.
- This provided a negative answer to Hilbert's Entscheidungsproblem.
- **Proof strategy:** We will try to show that if we had such a Total TM H , we would be able to build a Total TM for A_{TM}
- But since we proved that A_{TM} is undecidable, this would mean that **H is undecidable**.

The Halting problem

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$. Is $HALT_{TM}$ decidable?



Proof idea: We first assume that there exists such a Total Turing Machine H . Then, we use H as a subroutine to construct a Total Turing Machine A for A_{TM}



But A cannot be Total and so a total TM that decides H cannot exist

The Halting problem

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$. Is $HALT_{TM}$ decidable?



Proof idea: We first assume that there exists such a Total Turing Machine H . Then, we use H as a subroutine to construct a Total Turing Machine A for A_{TM}

Outlining the steps for building A using H :

- A calls $H(\langle M, w \rangle)$
- If H rejects, then we know that $M(w)$ loops forever and so A would output $REJECT$.



The Halting problem

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$. Is $HALT_{TM}$ decidable?



Proof idea: We first assume that there exists such a Total Turing Machine H . Then, we use H as a subroutine to construct a Total Turing Machine A for A_{TM}

Outlining the steps for building A using H :

- A calls $H(\langle M, w \rangle)$
- If H rejects, then we know that $M(w)$ loops forever and so A would output $REJECT$.
- If H accepts,
 - $M(w)$ surely halts (either accepts or rejects).



The Halting problem

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$. Is $HALT_{TM}$ decidable?



Proof idea: We first assume that there exists such a Total Turing Machine H . Then, we use H as a subroutine to construct a Total Turing Machine A for A_{TM}

Outlining the steps for building A using H :

- A calls $H(\langle M, w \rangle)$
- If H rejects, then we know that $M(w)$ loops forever and so A would output $REJECT$.
- If H accepts,
 - $M(w)$ surely halts (either accepts or rejects).
 - Simply run $M(w)$ and
 - $ACCEPT$ if $M(w)$ accepts
 - $REJECT$ if $M(w)$ rejects



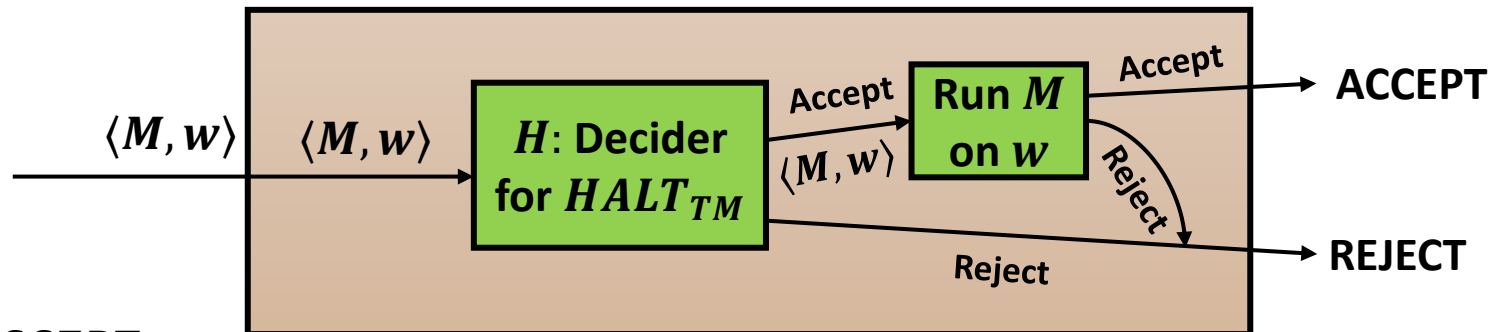
The Halting problem

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$. Is $HALT_{TM}$ decidable?

$$H(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ HALTS, i.e. accepts or rejects} \\ \text{REJECTS, if } M(w) \text{ does not HALT, i.e. loops infinitely} \end{cases}$$

Proof: Assume that there exists such a Total Turing Machine H . Then, we use H as a subroutine to construct a Total Turing Machine A for A_{TM} as follows:

$A =$ On input $\langle M, w \rangle$
Run $H(\langle M, w \rangle)$
If H rejects, output *REJECT*
If H accepts,
Run $M(w)$
If $M(w)$ accepts, output *ACCEPT*
If $M(w)$ rejects, output *REJECT*



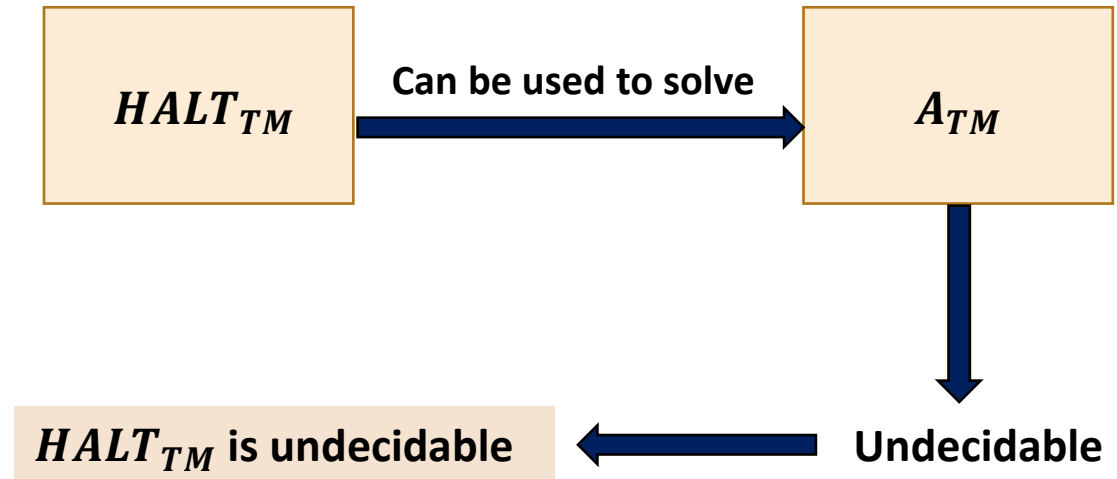
The Halting problem

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$. Is $HALT_{TM}$ decidable?

$$H(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ HALTS, i.e. accepts or rejects} \\ \text{REJECTS, if } M(w) \text{ does not HALT, i.e. loops infinitely} \end{cases}$$

Proof: Assume that there exists such a Total Turing Machine H . Then, we use H as a subroutine to construct a Total Turing Machine A for A_{TM} as follows:

$A =$ On input $\langle M, w \rangle$
Run $H(\langle M, w \rangle)$
If H rejects, output *REJECT*
If H accepts,
Run $M(w)$
If $M(w)$ accepts, output *ACCEPT*
If $M(w)$ rejects, output *REJECT*



The Halting problem

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$. Is $HALT_{TM}$ decidable?

$$H(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ HALTS, i.e. accepts or rejects} \\ \text{REJECTS, if } M(w) \text{ does not HALT, i.e. loops infinitely} \end{cases}$$

$HALT_{TM} \in RE$ as H halts whenever M accepts or rejects w and so

$Q =$ On input $\langle M, w \rangle$:

- Simulate M on input w
- If M accepts w , *ACCEPT*; if M rejects w , *ACCEPT*

Q recognizes $HALT_{TM}$

The Halting problem

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$. Is $HALT_{TM}$ decidable?

$$H(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ HALTS, i.e. accepts or rejects} \\ \text{REJECTS, if } M(w) \text{ does not HALT, i.e. loops infinitely} \end{cases}$$

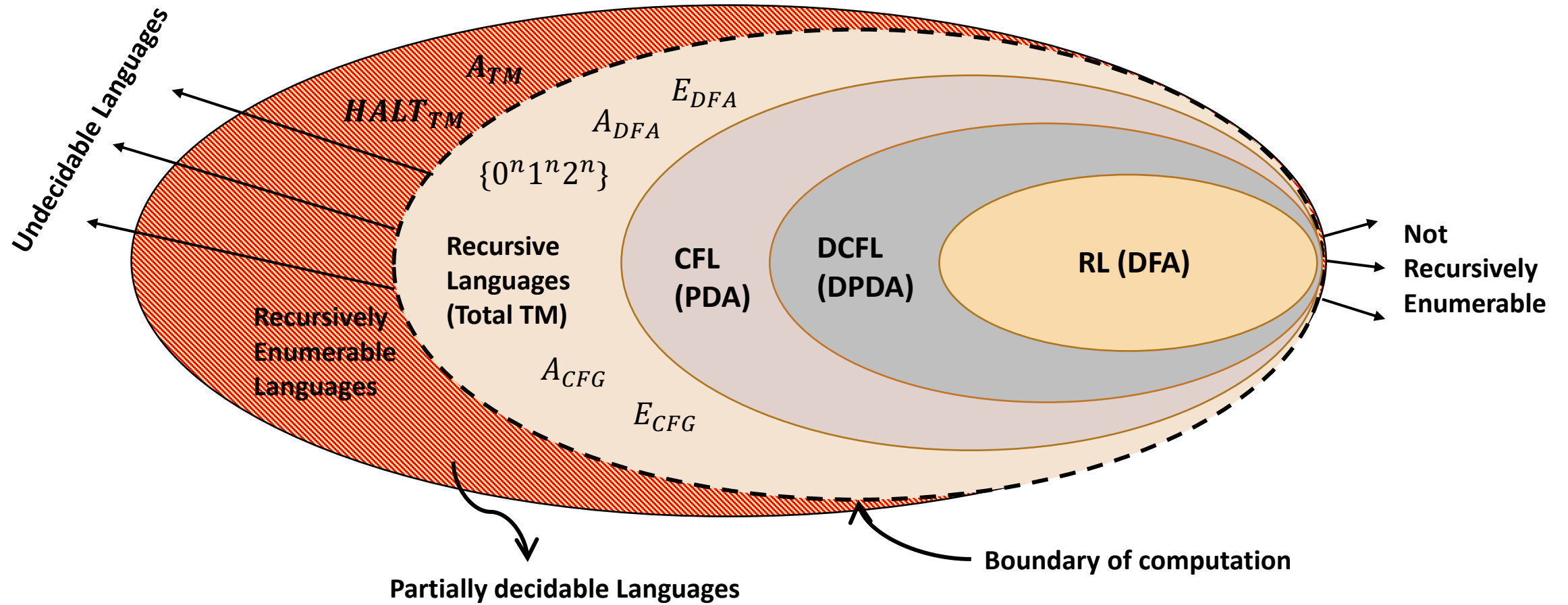
$HALT_{TM} \in RE$ as H halts whenever M accepts or rejects w and so

$Q =$ On input $\langle M, w \rangle$:

- Simulate M on input w
- If M accepts w , *ACCEPT*; if M rejects w , *ACCEPT*

Q recognizes $HALT_{TM}$

- $HALT_{TM}$ is undecidable
- $HALT_{TM} \in RE$ but not recursive
- $HALT_{TM}$ is partially decidable

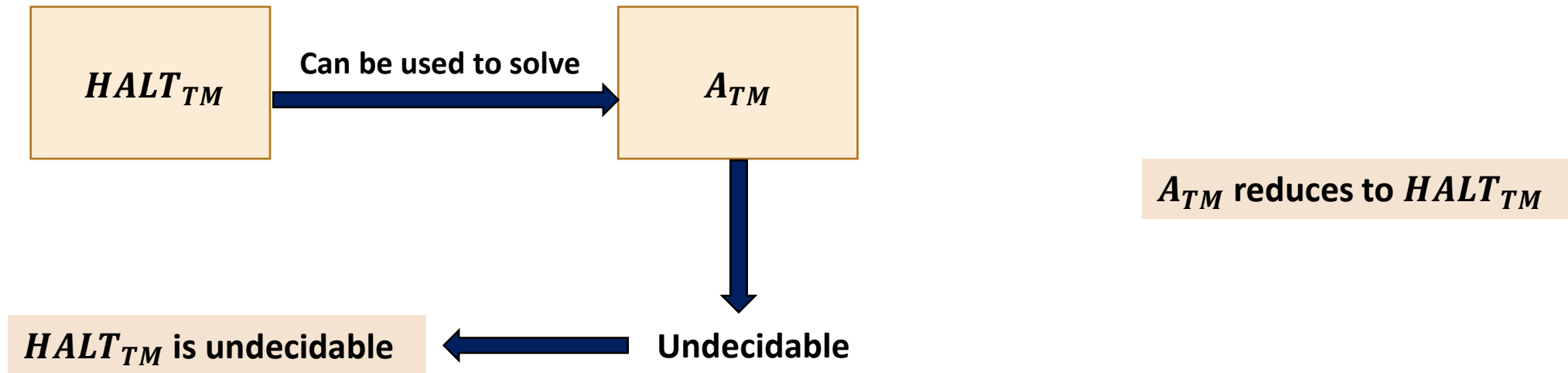


Reduction

Recall the proof of the undecidability of the Halting Problem

What did we do there?

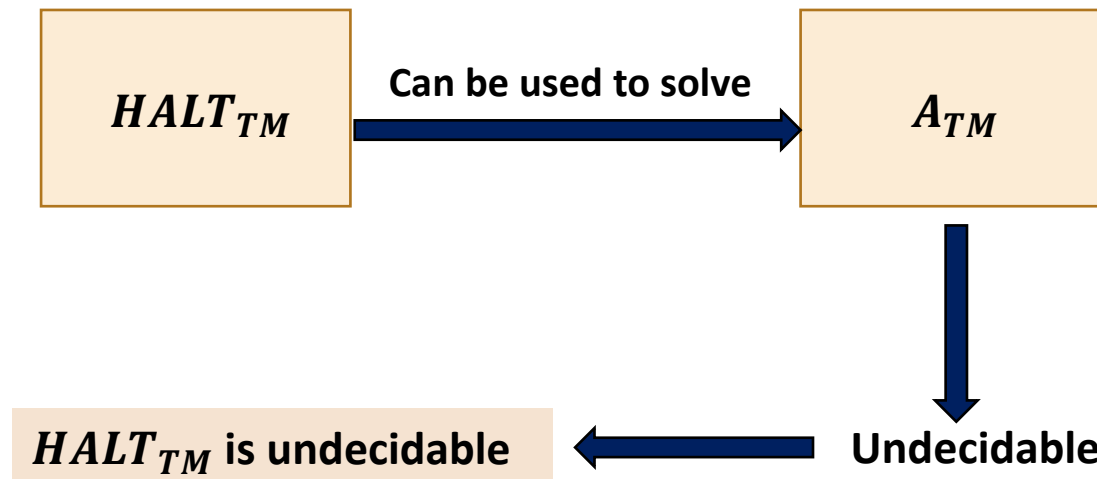
- We used a (supposed) decider for the Halting Problem to build a decider for A_{TM} .
- This established that $HALT_{TM}$ can be used to solve A_{TM} .
- As A_{TM} is undecidable, this established that $HALT_{TM}$ is undecidable too.
- The key underlying concept is an idea called **Reduction**.



Reduction

Generally,

- A language **A** **reduces to** another language **B** ($A \leq B$) iff we can **build a solver for A using a solver for B**
- In terms of computability, suppose using B we can compute A . Then, if **A is undecidable then so is B** .
- So, in the last proof we showed: $A_{TM} \leq HALT_{TM}$ to prove that $HALT_{TM}$ is undecidable.



Reduction

Generally,

- A language **A** **reduces to** another language **B** ($A \leq B$) iff we can **build a solver for A using a solver for B** .
- In terms of computability, suppose using B we can compute A . Then, if **A is undecidable then so is B** .
- So, in the last proof we showed: $A_{TM} \leq HALT_{TM}$ to prove that $HALT_{TM}$ is **undecidable**.
- This is a common technique to show that certain problems are decidable/undecidable.

Suppose, $A \leq B$ and

- **A is undecidable. Then B is undecidable.**

For example, we can prove that a problem P is undecidable by reducing the Halting problem to P .

$$HALT_{TM} \leq P \Rightarrow P \text{ is undecidable}$$

Reduction

Generally,

- A language **A** **reduces to** another language **B** ($A \leq B$) iff we can **build a solver for A using a solver for B** .
- In terms of computability, suppose using B we can compute A . Then, if **A is undecidable then so is B** .
- So, in the last proof we showed: $A_{TM} \leq HALT_{TM}$ to prove that $HALT_{TM}$ is **undecidable**.
- This is a common technique to show that certain problems are decidable/undecidable.

Suppose, $A \leq B$ and

- **A is undecidable. Then B is undecidable.**

For example, we can prove that a problem P is undecidable by reducing the Halting problem to P .

- Intuitively, this means **B is at least as hard as A** .
- So, if $A \notin R \Rightarrow B \notin R$

$HALT_{TM} \leq P \Rightarrow P$ is undecidable. Also, $P \notin R$

Reduction

Generally,

- A language **A** **reduces to** another language **B** ($A \leq B$) iff we can **build a solver for A using a solver for B** .
- In terms of computability, suppose using B we can compute A . Then, if **A is undecidable then so is B** .
- So, in the last proof we showed: $A_{TM} \leq HALT_{TM}$ to prove that $HALT_{TM}$ is **undecidable**.
- This is a common technique to show that certain problems are decidable/undecidable.

Suppose, $A \leq B$ and

- A is undecidable. Then B is undecidable.
- **B is decidable. Then A is decidable.**

- Intuitively, this means **B is at least as hard as A** .
- So, if $A \notin R \Rightarrow B \notin R$

- Intuitively, this means **A is not harder than B** .
- So, if $B \in R \Rightarrow A \in R$

Reduction

- A language **A** **reduces to** another language **B** ($A \leq B$) iff we can **build a solver for A using a solver for B** .
- If **A is undecidable then so is B** , i.e. **B is at least as hard as A** .
- If **B is decidable, then so is A** . Intuitively, this means **A is not harder than B** .
- This is a common technique to show that certain problems are decidable/undecidable.

This is how we can prove several problems are undecidable: by **reducing some known undecidable problem to these problems**.

Examples:

- $E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$
- $EQ_{TM} = \{\langle M_1, M_2 \rangle \mid M_1 \text{ and } M_2 \text{ are Turing Machines having } L(M_1) = L(M_2)\}$
- $ALL_{TM} = \{\langle M \rangle \mid M \text{ halts on all inputs}\}$
- $\vdots$

Reduction

This is how we can prove several problems are undecidable: by **reducing some known undecidable problem to these problems**.

Examples:

- $E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$
- $EQ_{TM} = \{\langle M_1, M_2 \rangle \mid M_1 \text{ and } M_2 \text{ are Turing Machines having } L(M_1) = L(M_2)\}$
- $ALL_{TM} = \{\langle M \rangle \mid M \text{ halts on all inputs}\}$
- $\vdots$

Generic strategy for proof:

- Consider that a Total TM for the given problem exists (say T_M).
- Build a total TM that can decide some undecidable problem (e.g. $HALT_{TM}$, A_{TM}) using T_M as a subroutine.

This reduction would prove that **E_{TM} , EQ_{TM} , ALL_{TM} are undecidable**.

Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N that decides $\overline{A_{TM}}$ using T_E .

What is $\overline{A_{TM}}$?

Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N that decides $\overline{A_{TM}}$ using T_E .

What is $\overline{A_{TM}}$?

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$$

$$\overline{A_{TM}} = \{\langle M, w \rangle \mid M \text{ DOES NOT accept input } w\}$$

Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N that decides $\overline{A_{TM}}$ using T_E .

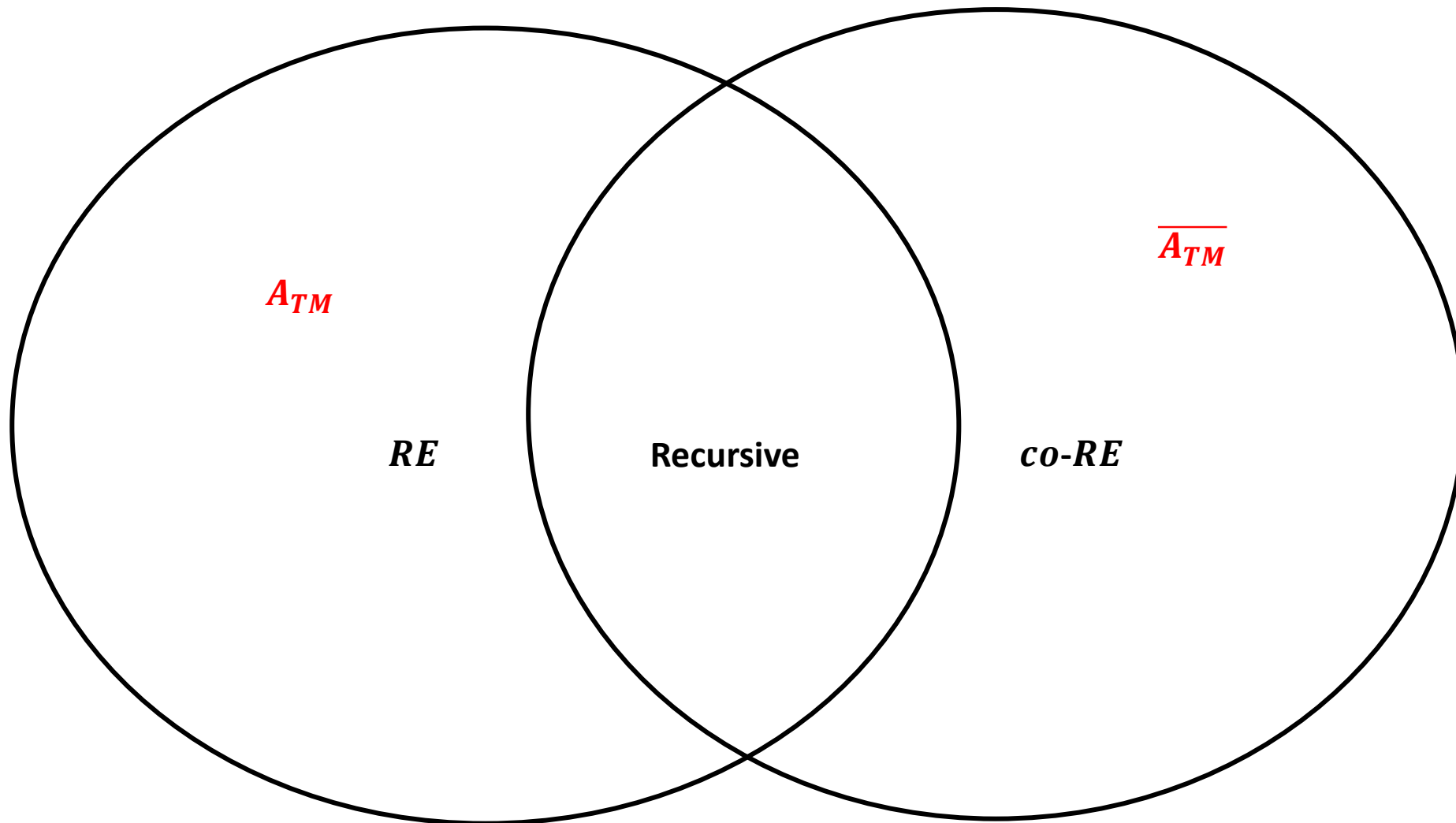
What is $\overline{A_{TM}}$?

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$$

$$\overline{A_{TM}} = \{\langle M, w \rangle \mid M \text{ DOES NOT accept input } w\}$$

$$A(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ accepts} \\ \text{REJECTS, if } M(w) \text{ rejects/loops} \end{cases}$$

$$N(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ rejects/loops} \\ \text{REJECTS, if } M(w) \text{ accepts} \end{cases}$$



Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N for $\overline{A_{TM}}$ using T_E .

What is $\overline{A_{TM}}$?

$\overline{A_{TM}} \in \text{co-RE} - R.$

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts input } w\}$$

$$\overline{A_{TM}} = \{\langle M, w \rangle \mid M \text{ DOES NOT accept input } w\}$$

$$A(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ accepts} \\ \text{REJECTS, if } M(w) \text{ rejects/loops} \end{cases}$$

$$N(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ rejects/loops} \\ \text{REJECTS, if } M(w) \text{ accepts} \end{cases}$$

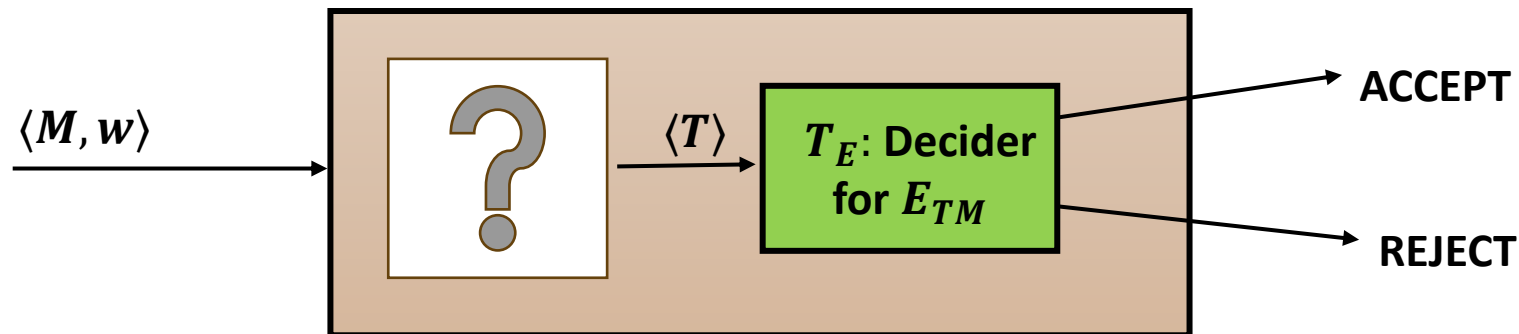
Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N for $\overline{A_{TM}}$ using T_E .

$$N(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ rejects/loops} \\ \text{REJECTS, if } M(w) \text{ accepts} \end{cases}$$

$$T_E(\langle T \rangle) = \begin{cases} \text{ACCEPTS, if } L(\langle T \rangle) = \Phi \\ \text{REJECTS, if } L(\langle T \rangle) \neq \Phi \end{cases}$$



Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

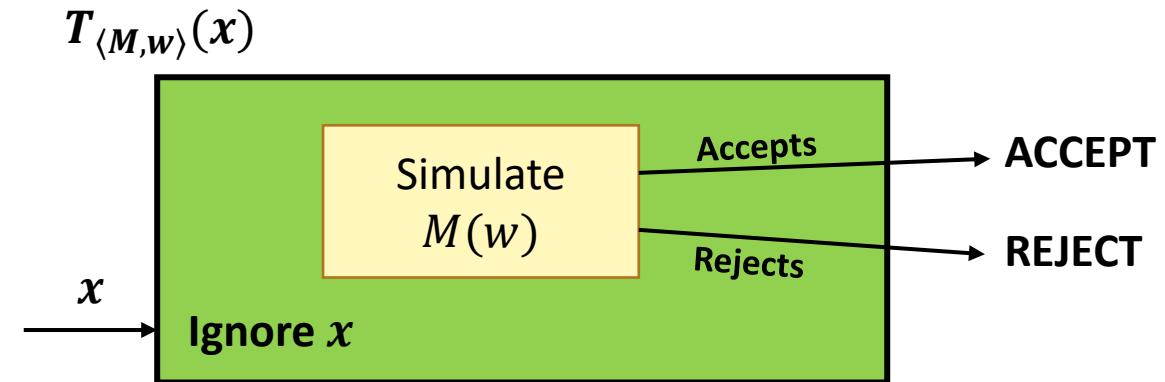
Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N for $\overline{A_{TM}}$ using T_E .

$$N(\langle M, w \rangle) = \begin{cases} \text{ACCEPTS, if } M(w) \text{ rejects/loops} \\ \text{REJECTS, if } M(w) \text{ accepts} \end{cases}$$

$$T_E(\langle T \rangle) = \begin{cases} \text{ACCEPTS, if } L(\langle T \rangle) = \Phi \\ \text{REJECTS, if } L(\langle T \rangle) \neq \Phi \end{cases}$$

Key idea:

- $N(\langle M, w \rangle)$ first builds the encoding of a TM $T_{\langle M, w \rangle}$.
- $T_{\langle M, w \rangle}$ does not accept any string if and only if $M(w)$ does not accept.
- That is, running $T_{\langle M, w \rangle}(x)$ on **ANY** input x , runs M on w .



What does this achieve??

Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

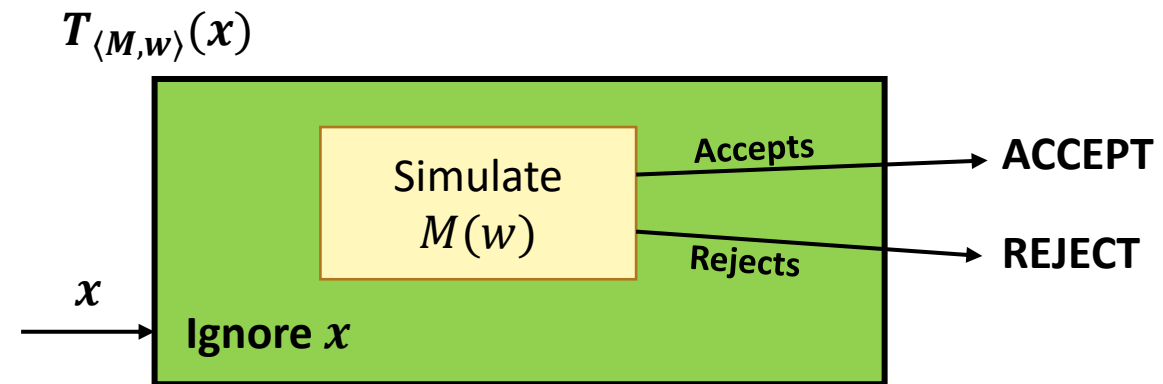
Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N for $\overline{A_{TM}}$ using T_E .

Key idea:

- $N(\langle M, w \rangle)$ first builds the encoding of a TM $T_{\langle M, w \rangle}$
- $T_{\langle M, w \rangle}$ does not accept any string if and only if M does not accept w .
- That is, running $T_{\langle M, w \rangle}(x)$ on **ANY** input x , runs M on w .

What does this achieve??

- This means, $L(T) = \Phi$ if M does not accept w and $L(T) \neq \Phi$ if M accepts w !
- This allows N to call $T_E(\langle T \rangle)$



Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N for $\overline{A_{TM}}$ using T_E .

Key idea:

- $N(\langle M, w \rangle)$ first builds the encoding of a TM $T_{\langle M, w \rangle}$
- Running $T_{\langle M, w \rangle}(x)$ on **ANY** input x , runs M on w .
- $T_{\langle M, w \rangle}$ does not accept any input x if and only if M does not accept w .
- This means, $L(T) = \Phi$ if M does not accept w and $L(T) = \Sigma^*$ if M accepts w !
- This allows N to call $T_E(\langle T \rangle)$ and
 - ACCEPT if $T_E(\langle T \rangle)$ accepts $\Leftrightarrow L(T) = \Phi$ [M does not accept w]
 - REJECT if $T_E(\langle T \rangle)$ rejects $\Leftrightarrow L(T) \neq \Phi$ [M accepts w]

Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N for $\overline{A_{TM}}$ using T_E .

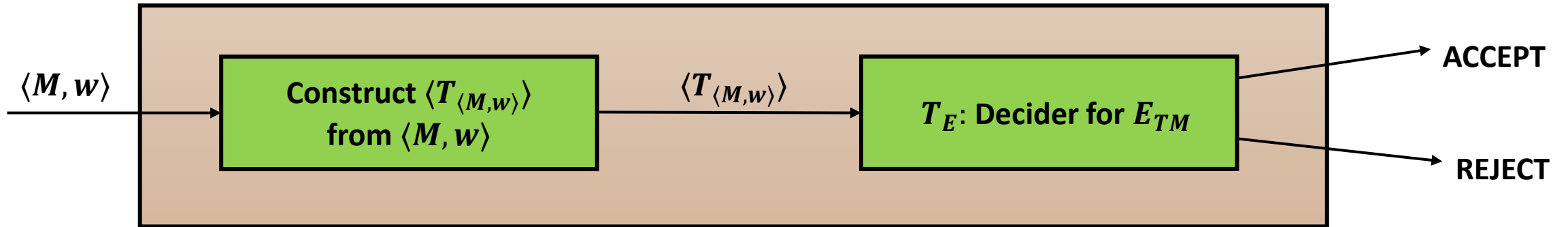
Key idea:

- $N(\langle M, w \rangle)$ first builds the encoding of a TM $T_{\langle M, w \rangle}$
- Running $T_{\langle M, w \rangle}(x)$ on **ANY** input x , runs M on w .
- $T_{\langle M, w \rangle}$ does not accept any input x if and only if M does not accept w .
- This means, $L(T) = \Phi$ if M does not accept w and $L(T) = \Sigma^*$ if M accepts w !
- This allows N to call $T_E(\langle T \rangle)$ and
 - ACCEPT if $T_E(\langle T \rangle)$ accepts $\Leftrightarrow L(T) = \Phi$ [M does not accept w]
 - REJECT if $T_E(\langle T \rangle)$ rejects $\Leftrightarrow L(T) \neq \Phi$ [M accepts w]
- **Deciding $\overline{A_{TM}}$ is tied to whether $L(T) = \Phi$!**

Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

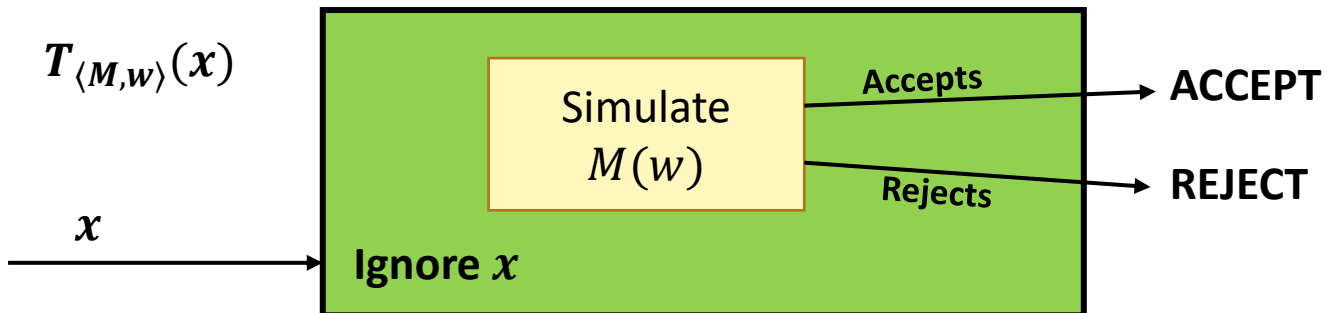
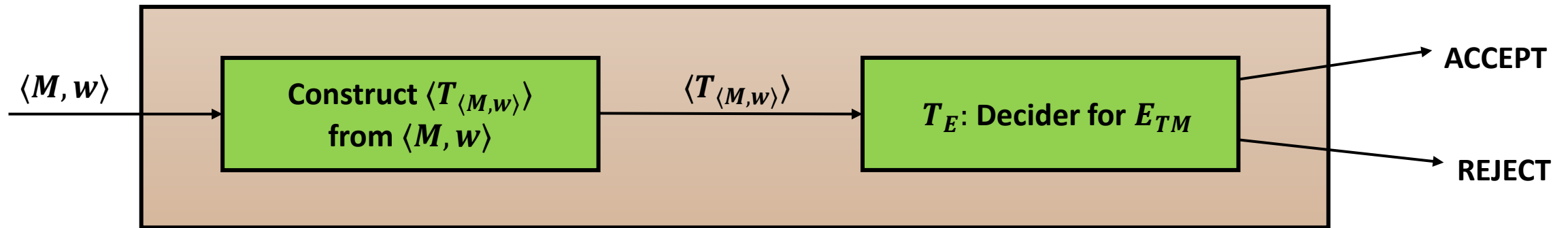
Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N for $\overline{A_{TM}}$ using T_E .



Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N for $\overline{A_{TM}}$ using T_E .



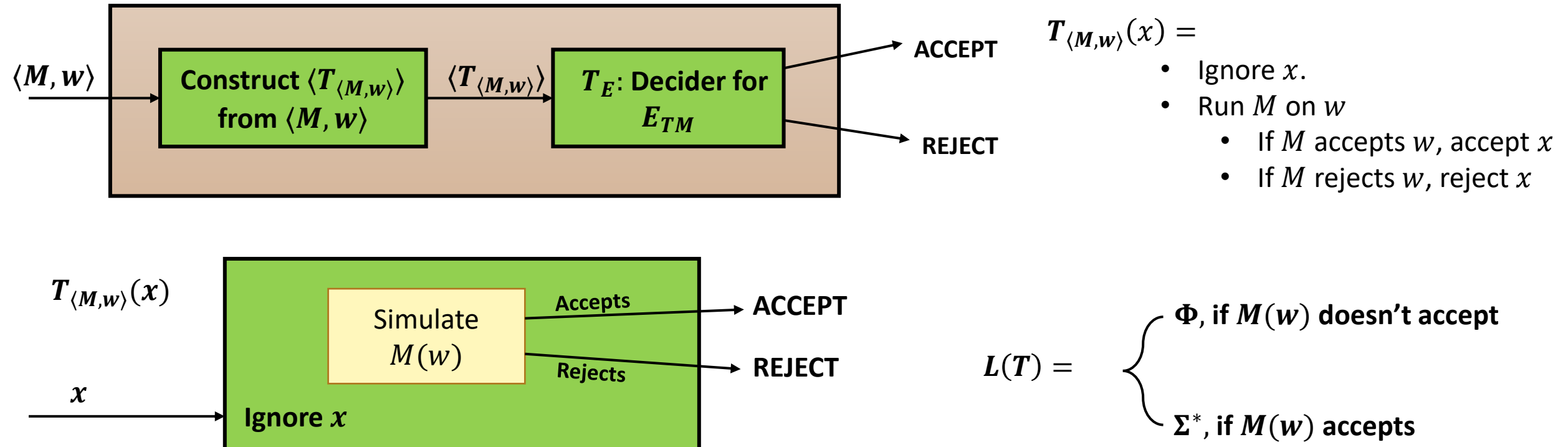
$$T_{\langle M, w \rangle}(x) =$$

- Ignore x .
- Run M on w
 - If M accepts w , accept x
 - If M rejects w , reject x

Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

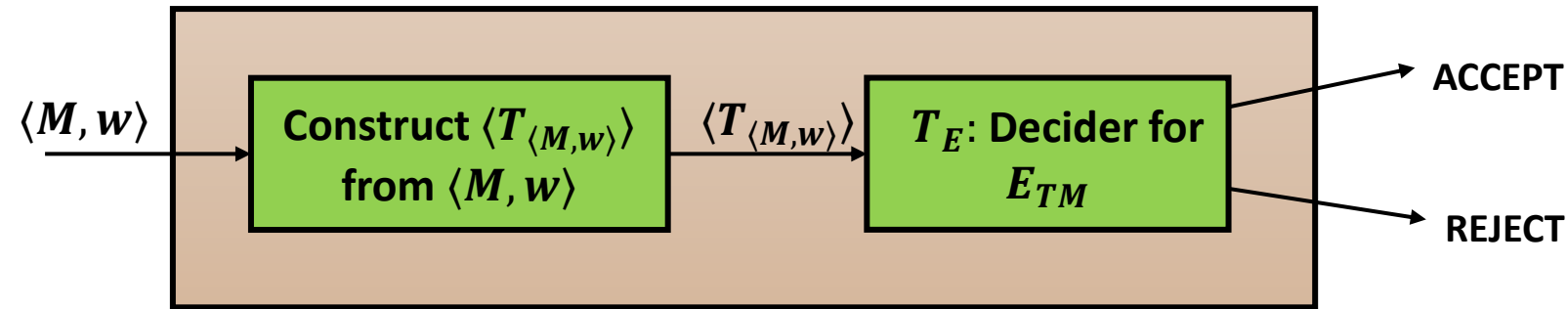
Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N for $\overline{A_{TM}}$ using T_E .



Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N for $\overline{A_{TM}}$ using T_E .



$$T_{\langle M, w \rangle}(x) =$$

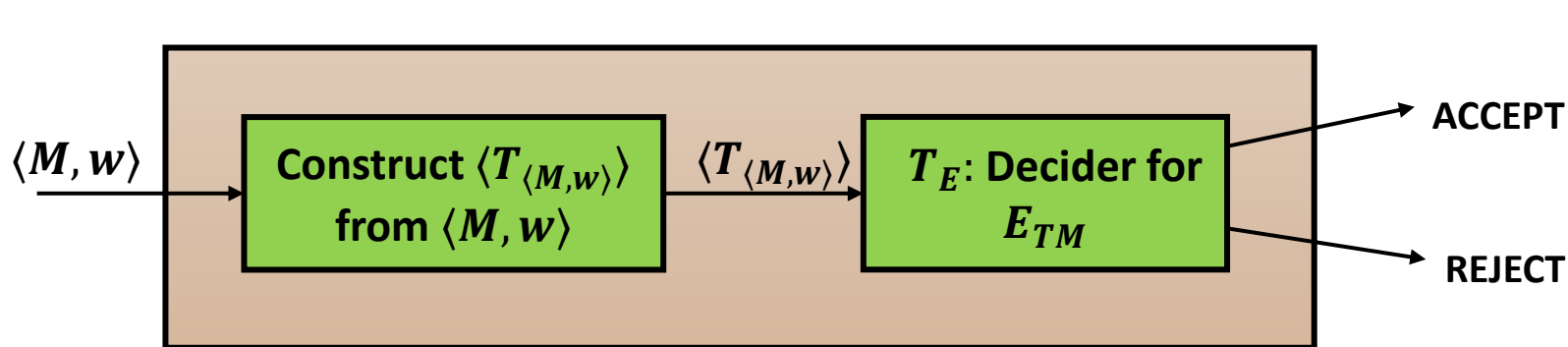
- Ignore x .
- Run M on w
 - If M accepts w , accept x
 - If M rejects w , reject x

$$L(T) = \begin{cases} \Phi, & \text{if } M(w) \text{ doesn't accept} \\ \Sigma^*, & \text{if } M(w) \text{ accepts} \end{cases}$$

Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N that decides $\overline{A_{TM}}$ using T_E .



$$T_{\langle M, w \rangle}(x) =$$

- Ignore x .
- Run M on w
 - If M accepts w , accept x
 - If M rejects w , reject x

$$L(T) = \begin{cases} \Phi, & \text{if } M(w) \text{ doesn't accept} \\ \Sigma^*, & \text{if } M(w) \text{ accepts} \end{cases}$$

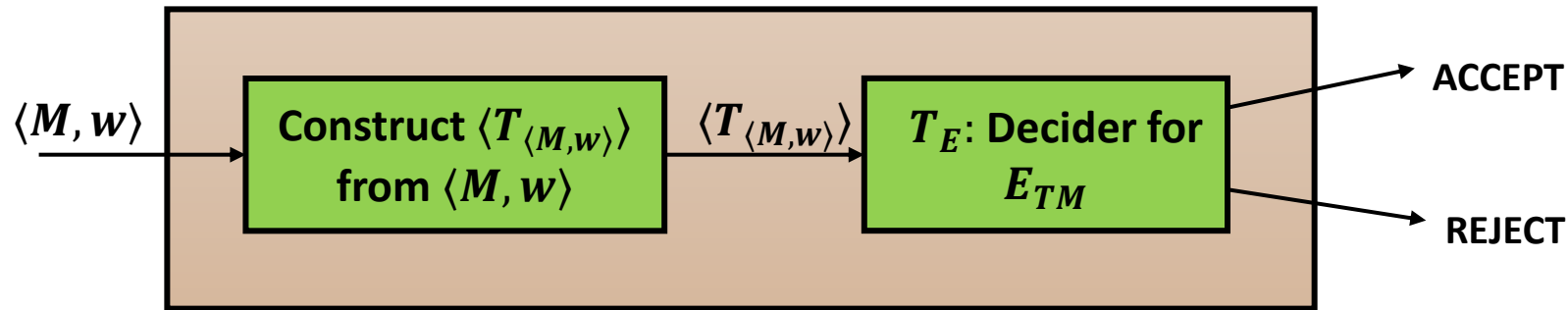
$$T_E(\langle T \rangle) =$$

- ACCEPT, if $L(T) = \Phi$
- REJECT, if $L(T) \neq \Phi$

Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N that decides $\overline{A_{TM}}$ using T_E .



$$T_{\langle M, w \rangle}(x) =$$

- Ignore x .
- Run M on w
 - If M accepts w , accept x
 - If M rejects w , reject x

$$L(T) = \begin{cases} \Phi, & \text{if } M(w) \text{ doesn't accept} \\ \Sigma^*, & \text{if } M(w) \text{ accepts} \end{cases}$$

$$T_E(\langle T \rangle) =$$

- ACCEPT, if $L(T) = \Phi$ (if $M(w)$ doesn't accept)
- REJECT, if $L(T) \neq \Phi$ (if $M(w)$ accepts)

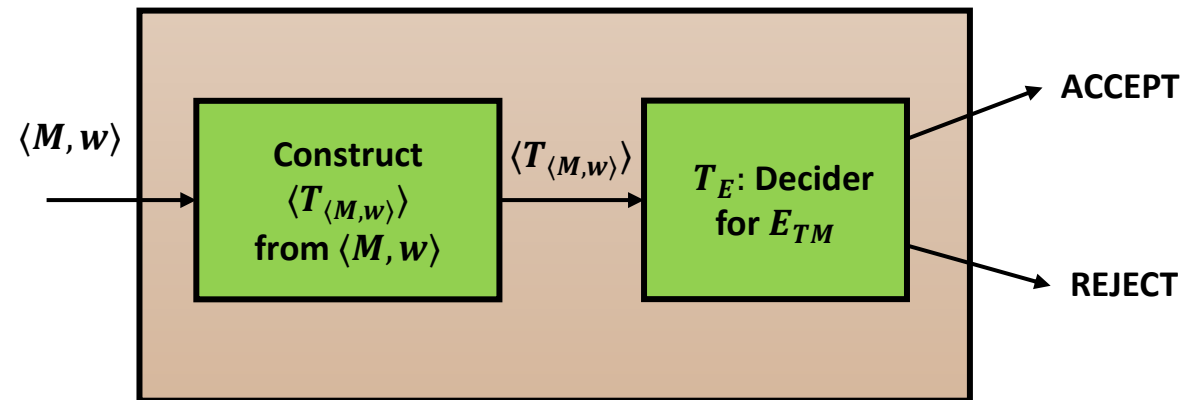
Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N that decides $\overline{A_{TM}}$ using T_E .

$$N(\langle M, w \rangle) =$$

- Construct $\langle T_{\langle M, w \rangle} \rangle$, the encoding of $T_{\langle M, w \rangle}$ such that for any input x it works as follows:
 - Ignore x .
 - Run M on w
 - If M accepts w , accept x
 - If M rejects w , reject x
- Send $\langle T_{\langle M, w \rangle} \rangle$ to T_E and Output
ACCEPT if T_E accepts
REJECT if T_E rejects



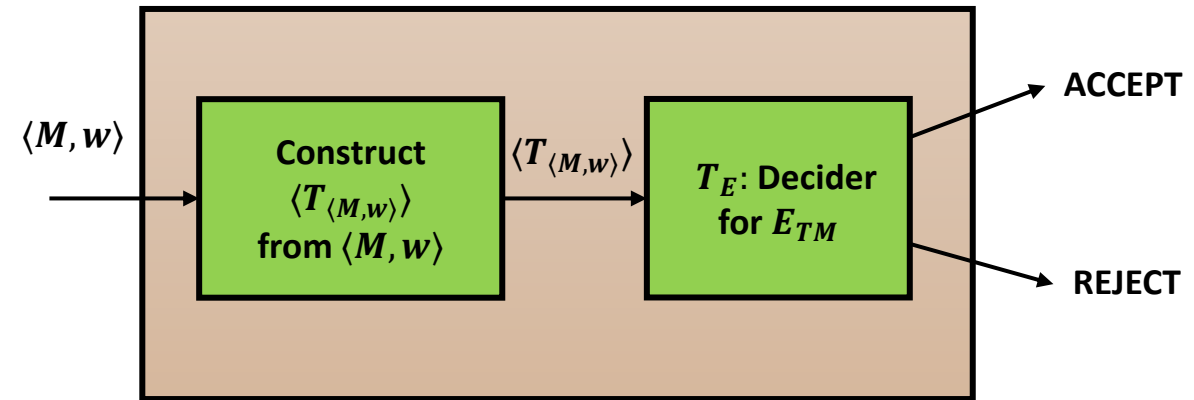
Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N that decides $\overline{A_{TM}}$ using T_E .

$N(\langle M, w \rangle) =$

- Construct $\langle T_{\langle M, w \rangle} \rangle$, the encoding of $T_{\langle M, w \rangle}$ such that for any input x it works as follows:
 - Ignore x .
 - Run M on w
 - If M accepts w , accept x
 - If M rejects w , reject x
- Send $\langle T_{\langle M, w \rangle} \rangle$ to T_E and Output
ACCEPT if T_E accepts
REJECT if T_E rejects

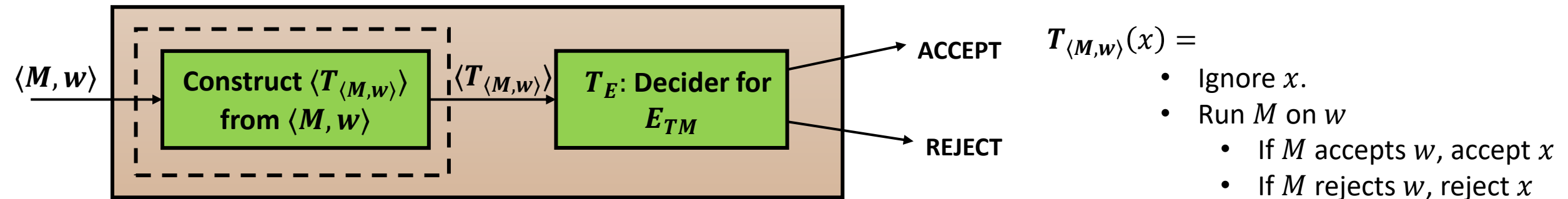


- $\overline{A_{TM}} \leq E_{TM}$
- $\overline{A_{TM}}$ is undecidable
- E_{TM} is undecidable!

Undecidability

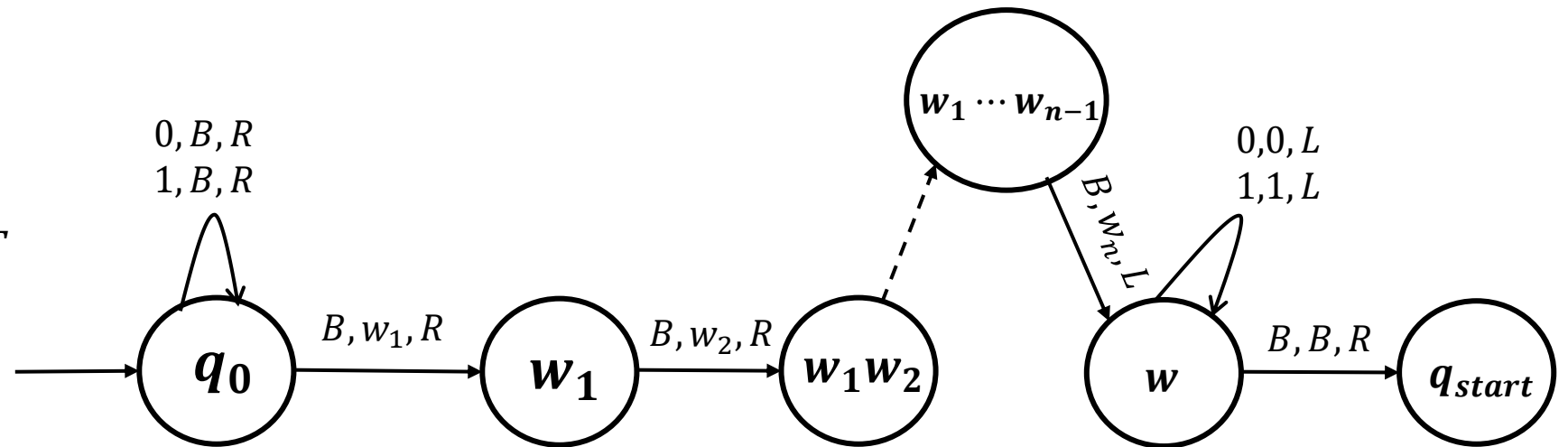
$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N that decides $\overline{A_{TM}}$ using T_E .



$T_{\langle M, w \rangle}$ from $\langle M, w \rangle$:

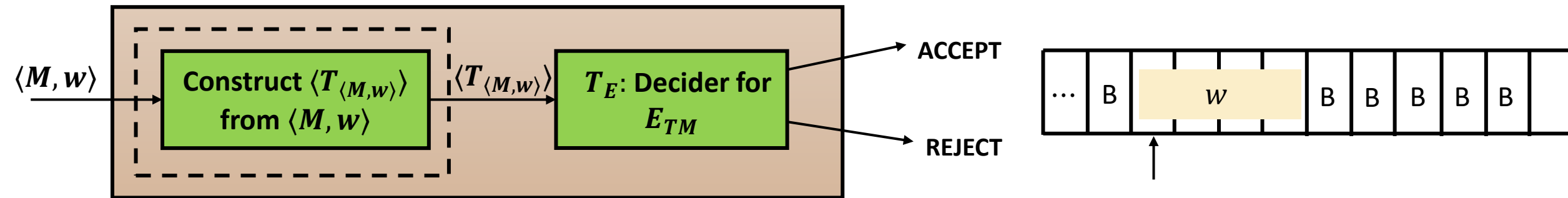
- Remove i/p x from the tape of T
- 'Embed' w into the TM of M



Undecidability

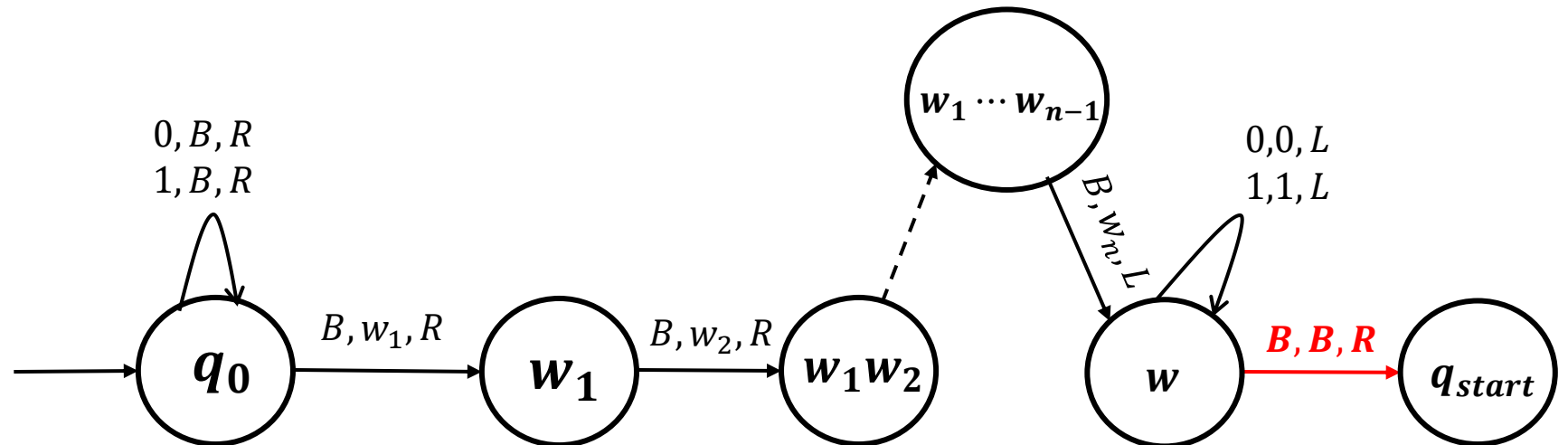
$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N that decides $\overline{A_{TM}}$ using T_E .



$T_{\langle M, w \rangle}$ from $\langle M, w \rangle$:

- Remove i/p x from the tape of T
- 'Embed' w into the TM of M



Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N that decides $\overline{A_{TM}}$ using T_E .

$$N(\langle M, w \rangle) =$$

- Construct $\langle T_{\langle M, w \rangle} \rangle$, the encoding of $T_{\langle M, w \rangle}$ such that for any input x it works as follows:
 - Ignore x .
 - Run M on w
 - If M accepts w , accept x
 - If M rejects w , reject x
- Send $\langle T_{\langle M, w \rangle} \rangle$ to T_E and Output
ACCEPT if T_E accepts
REJECT if T_E rejects

- $\overline{A_{TM}} \leq E_{TM}$
- $\overline{A_{TM}}$ is undecidable
- E_{TM} is undecidable!

Claim: $E_{TM} \in \text{co-RE} - R$

WHY?

Undecidability

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi\}$$

Proof: Let T_E be the Turing Machine that decides E_{TM} . We shall prove that $\overline{A_{TM}} \leq E_{TM}$ by constructing a Turing Machine N that decides $\overline{A_{TM}}$ using T_E .

$$N(\langle M, w \rangle) =$$

- Construct $\langle T_{\langle M, w \rangle} \rangle$, the encoding of $T_{\langle M, w \rangle}$ such that for any input x it works as follows:
 - Ignore x .
 - Run M on w
 - If M accepts w , accept x
 - If M rejects w , reject x
- Send $\langle T_{\langle M, w \rangle} \rangle$ to T_E and Output
ACCEPT if T_E accepts
REJECT if T_E rejects

- $\overline{A_{TM}} \leq E_{TM}$
- $\overline{A_{TM}}$ is undecidable
- E_{TM} is undecidable!

$$E_{TM} \in \mathbf{co-RE-R}$$

Proof idea: We can build a co-recognizer for E_{TM} .

$C =$ On input $\langle M \rangle$

- For $i = 1, 2, 3, \dots$
 - For $j = 1, 2, 3, \dots i$
Run M on s_j for i steps.
If M accepts s_j , REJECT.

Thank You!